



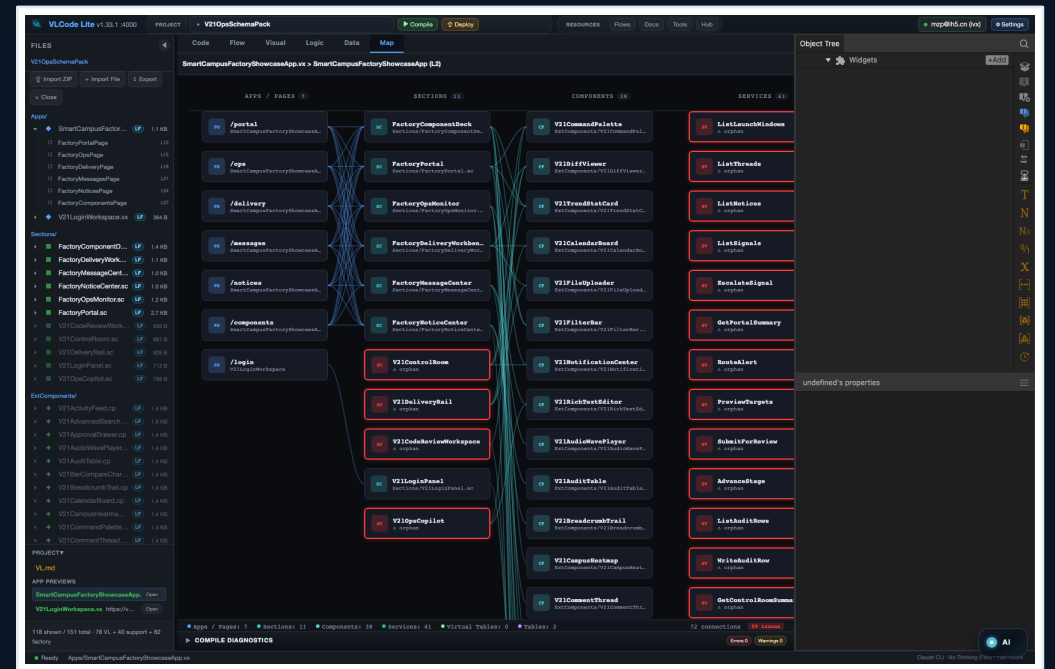
VL PROGRAMMING LANGUAGE · VLC LOCAL IDE

A language built for AI — and a DAG that tames it.

VL redesigns how software is represented for AI. VLC turns the entire build process into a graph you can see, verify, and trust.

Open source (MIT) · Local-first · Export anytime

github.com/VisualLogic-AI/VL-Code



Map mode — an entire VL app as one architecture graph.

01 THE PROBLEM

AI can ace a 'point'. It can't hold the 'line'.

The bottleneck is no longer model intelligence. It is how code is represented and how the process is controlled.

Languages were never built for AI

They assume a human author + a compiler. AI is left to guess the hidden structure.

Agents drift off course

Point-by-point I/O wanders — you babysit every step, and still miss.

Expensive, expert-only

Tokens burned regenerating the same code — that you still have to read.

VL changes the representation. VLC changes the process.

VL is a language designed for AI — not low-code, not a DSL.

General-purpose, with no capability ceiling. It works with today's models — even small, cheap, local ones.

AI-native

Language, IDE and AI designed together from day one.

General-purpose

No ceiling — real apps, front to back, not toy flows.

Stable today

Models already generate VL reliably. No 'wait 5 years'.

Low-code hides complexity. VL structures it.

Decoupled on every axis — so generation runs in parallel.

Front/back · look/structure · build/deploy · data/usage. Each file is small, single-purpose, and standalone.

Front / Back

.sc + .cp / .vs

Look / Structure

THEME / components

Build / Deploy

source / target

Data / Usage

.vdb / VirtualTable

High-concurrency generation

Services and front-end modules generate in parallel, not one long file.

Automatic assembly

Clean contracts let the factory snap components together.

Small files · single responsibility → small context, parallel, fewer hallucinations.

Everything is a component — front-end and back-end alike.

The minimal unit of a VL program. The shape an AI agent reasons about best.



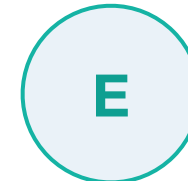
Properties

what it holds



Methods

what it does



Events

what it reacts to

1D — lines of text



2D — a graph of components

A program is a composition of components. Structured, decoupled — exactly how an Agent works.

The Component Factory: 16,000+ reusable building blocks.

Not just UI snippets — installable contracts for real VL projects. The expensive token-compute, banked.

16,000+

components

P+M+E

every interface

0

manual glue

÷10

tokens

Expensive generation happens once. Everyone reuses it forever.

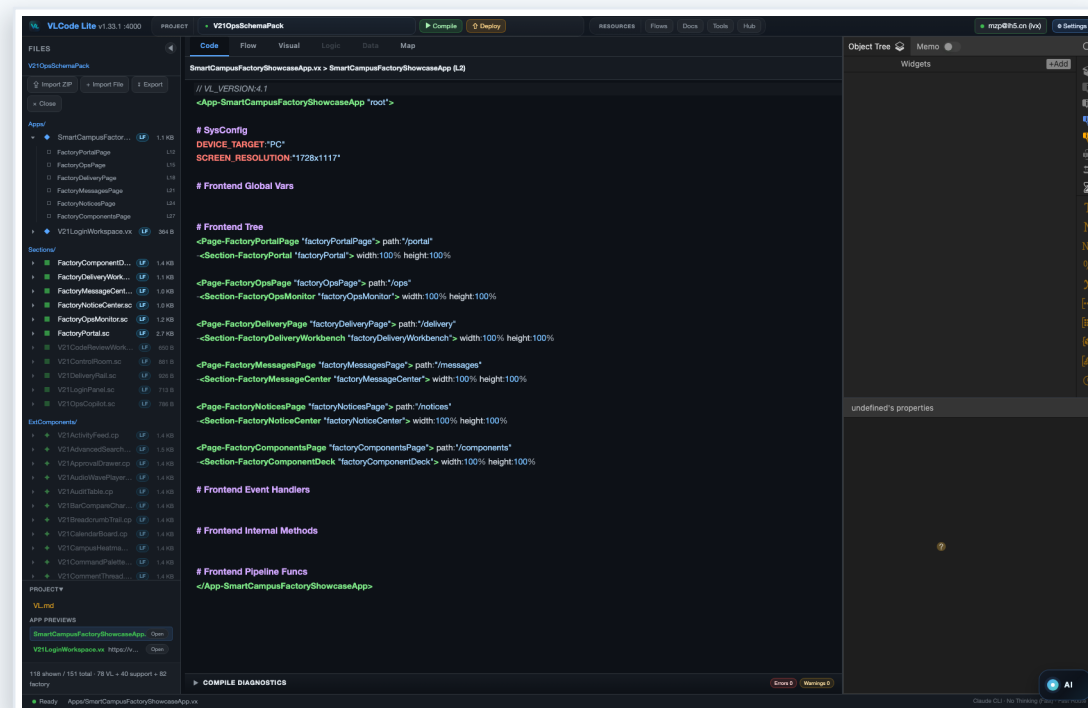
VLC: a local IDE built for VL + AI.

As familiar as VS Code — and designed end-to-end for the VL language and AI. Local-first: your machine, your code, your models.

- Local read/write + AI — the smoothest path

- Cloud optional, pay-as-you-go

- Zero lock-in, export anytime

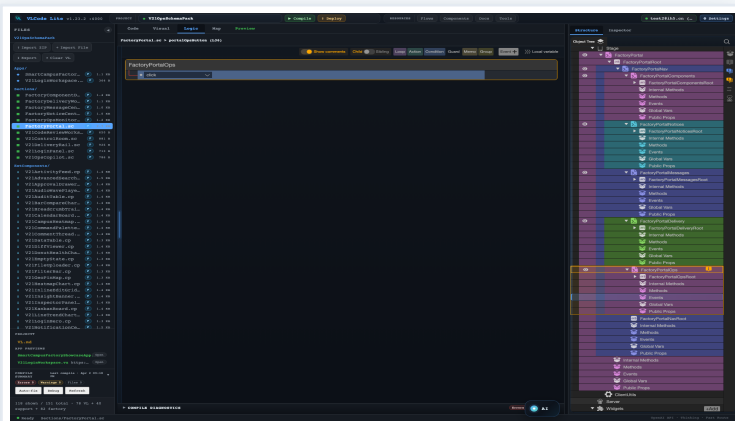


Code mode — VL and generated source, side by side.

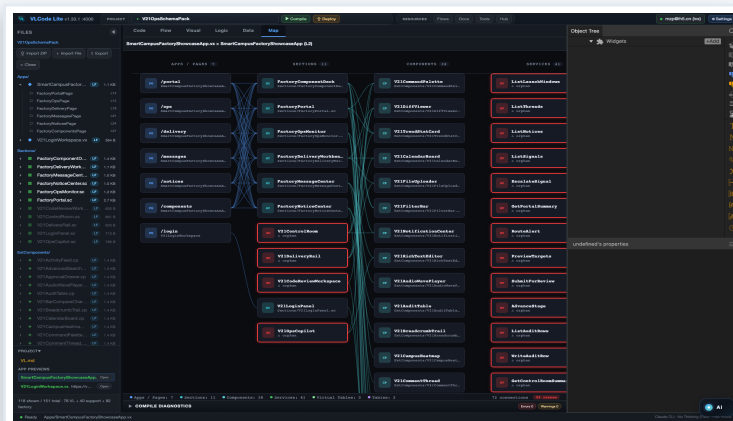
Local-first. Cloud optional. The whole stack on your machine.

See everything: logic, structure, and process.

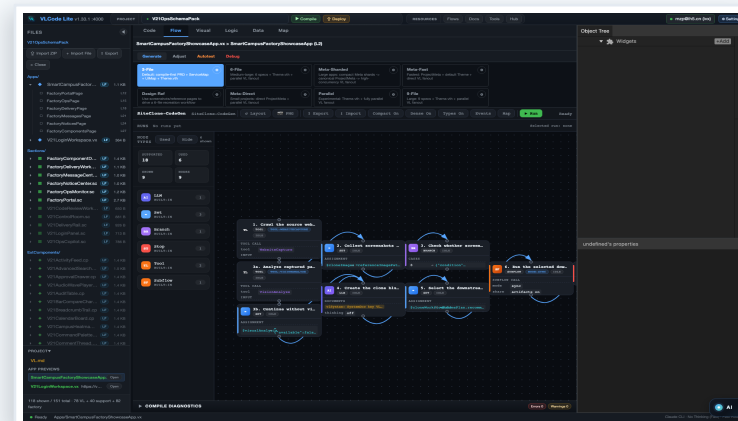
Every internal detail of a VL app is projected as a picture — and code ↔ graph stays 1:1, both directions.



Event Panel — internal logic



Map — layered call graph

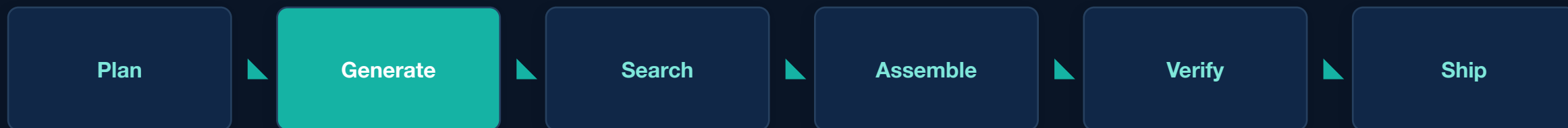


DAG — the generation process

You reason in pictures. The IDE is a projection of the language.

DAG is the core of VLC — and the tool that tames AI.

Every agent works around the DAG. Creativity lives inside the nodes; determinism lives in the edges.



Inside nodes = creativity

Between edges = determinism

AI stays powerful — the process stays controllable.

DAG vs. today's agents.

Most agents are either too complex (you must code them) or a single I/O point. The DAG makes the whole agent process visible and programmable.

Today's agents

- ✗ Too complex to wire — or just a single I/O point
- ✗ A black box that drifts off course
- ✗ "Looks right" — hard to verify
- ✗ Output you can't predict

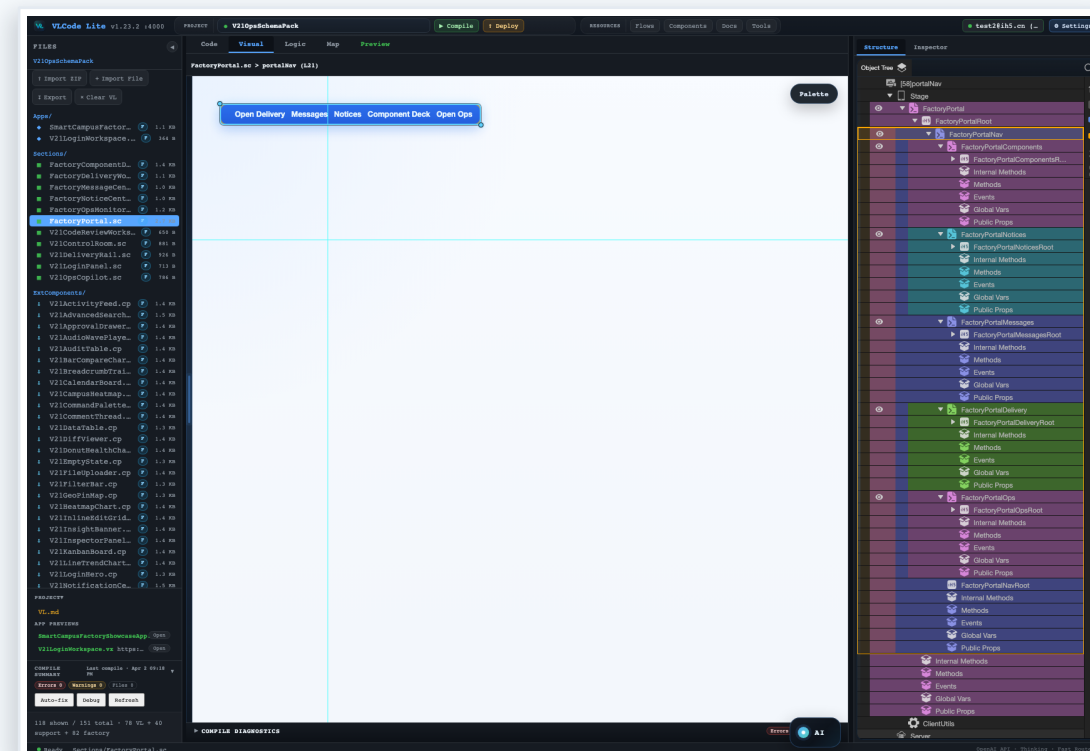
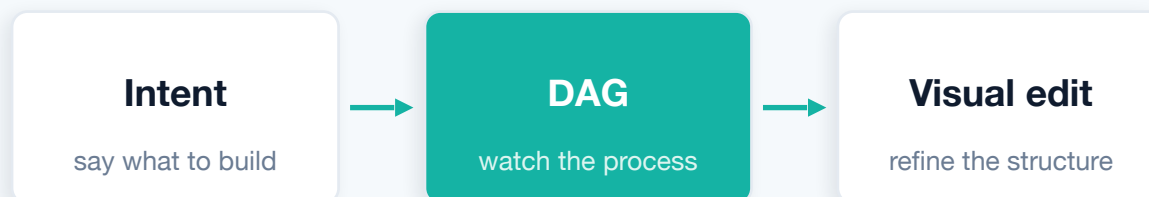
VLC's DAG

- ✓ Visual + programmable orchestration, zero coding
- ✓ Always in bounds — fully controllable
- ✓ Every node can lint / compile / preview / test
- ✓ Produces exactly what the designer intended

All of AI's power, kept inside the lines — technically ahead of point-wise agents.

Build software with zero programming knowledge.

Say what you want, watch the DAG run, refine visually. Code is still there — but it's no longer the only doorway in.

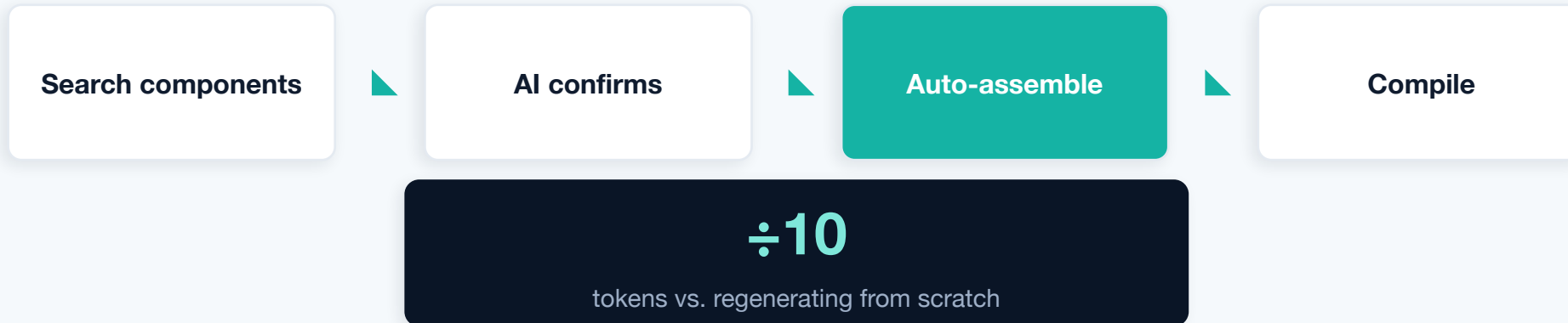


Visual Panel — inspect and adjust the app as structure.

For the first time, the door to programming opens to everyone.

Search-and-assemble nodes: store the compute, don't burn it.

Drop a 'search components' node into the DAG. Every match is fetched, the AI confirms, and they snap together automatically.



Intelligence is banked in components, in the cloud — loaded on demand, not re-billed.

Nothing is a black box — bring your own.

Every resource layer is open for you to extend. Upload it, version it, reuse it across projects.



Workflows

your own DAGs



Docs

specs & context



Tools

custom actions



Components

into the Factory



Skills

agent capabilities

Your workflows, your components, your tools — all uploadable, all yours.

Export, self-host, open source — zero lock-in.

What you build is yours, with no limits on where it runs and no walls around the core technology.

Export

Apps export to React + Node source — run them anywhere.

Self-host

Private deployment with no restrictions whatsoever.

Open source

VL syntax and core docs are open. Every component is open.

Generate once, own it forever. The syntax and every component are open.

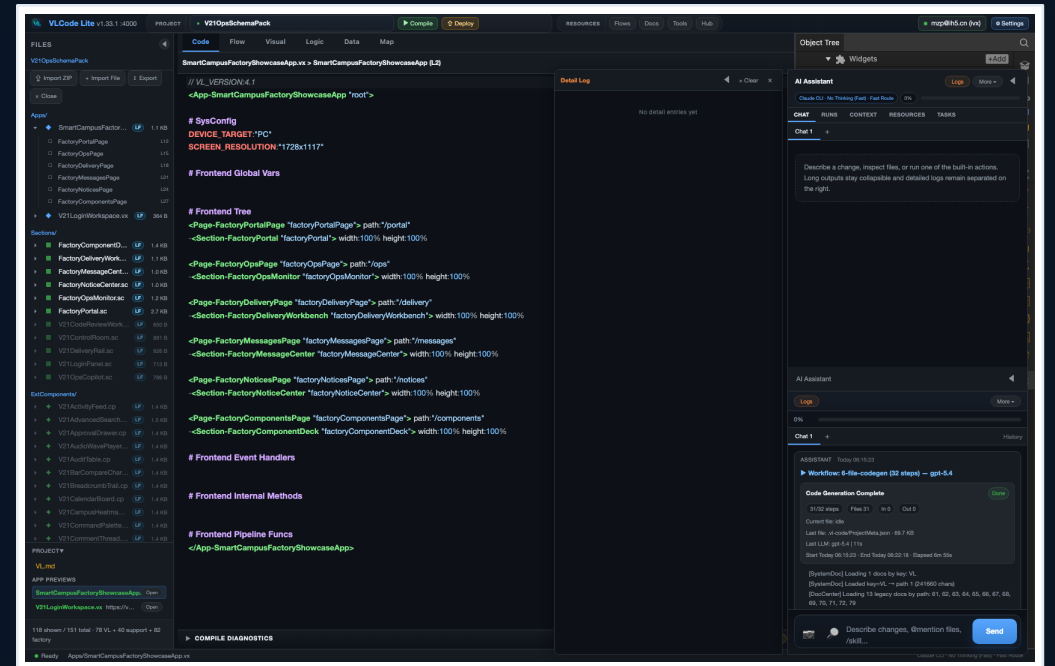
WHAT WE SOLVE

VLC solves the control problem of AI software generation.

VL gives AI the right representation.

DAG gives AI the right process.

The Factory gives AI reusable engineering memory.



Chat + project context + DAG execution — in one local IDE.

github.com/VisualLogic-AI/VL-Code